

<HLS Programming with FPGAs>

The HLS Book

Chapter 2

Finite Impulse Response (FIR) Filters 2

Choi, Young-kyu

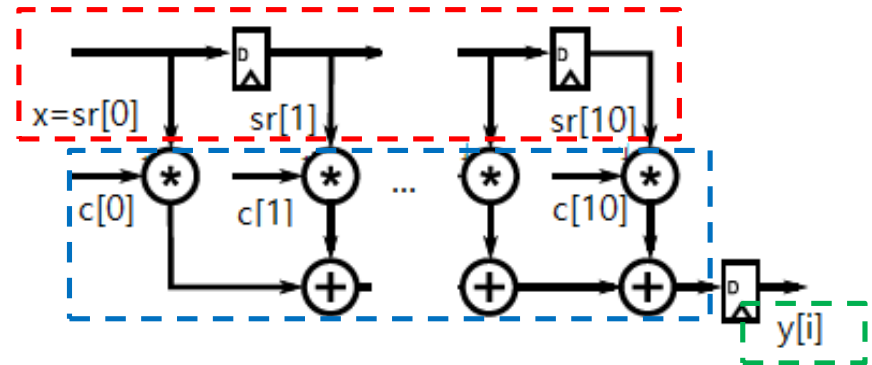
Recap: Baseline FIR filter code

```
#define N 11  —————> 11 taps filter
#include "ap_int.h"
```

```
typedef int coef_t;
typedef int data_t;
typedef int acc_t;
```

```
void fir(data_t *y, data_t x) {
    coef_t c[N] = {
        53, 0, -91, 0, 313, 500, 313, 0, -91, 0, 53
    };
    static
    data_t shift_reg[N];
    acc_t acc;
    int i;
```

*MAC: multiply and accumulate



```
    acc = 0;
    Shift_Accum_Loop:
    for (i = N - 1; i >= 0; i--) {
        if (i == 0) {
            acc += x * c[0];
            shift_reg[0] = x;
        } else {
            shift_reg[i] = shift_reg[i - 1];
            acc += shift_reg[i] * c[i];
        }
    }
    *y = acc;
```

Learning Objectives

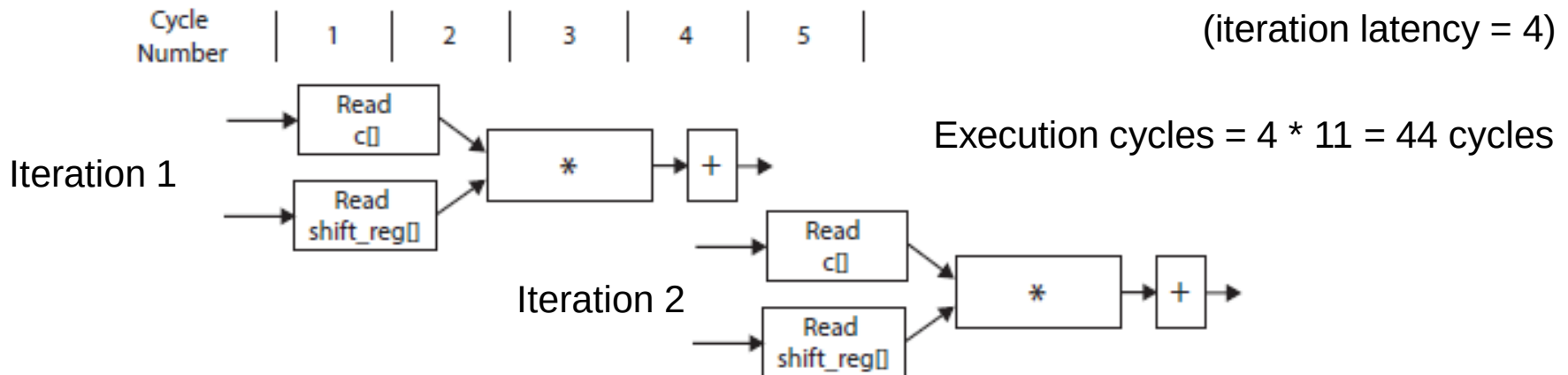
- Optimize the baseline FIR filter with HLS compiler directives and code refactoring

2.9 Loop Pipelining

- Baseline FIR code:

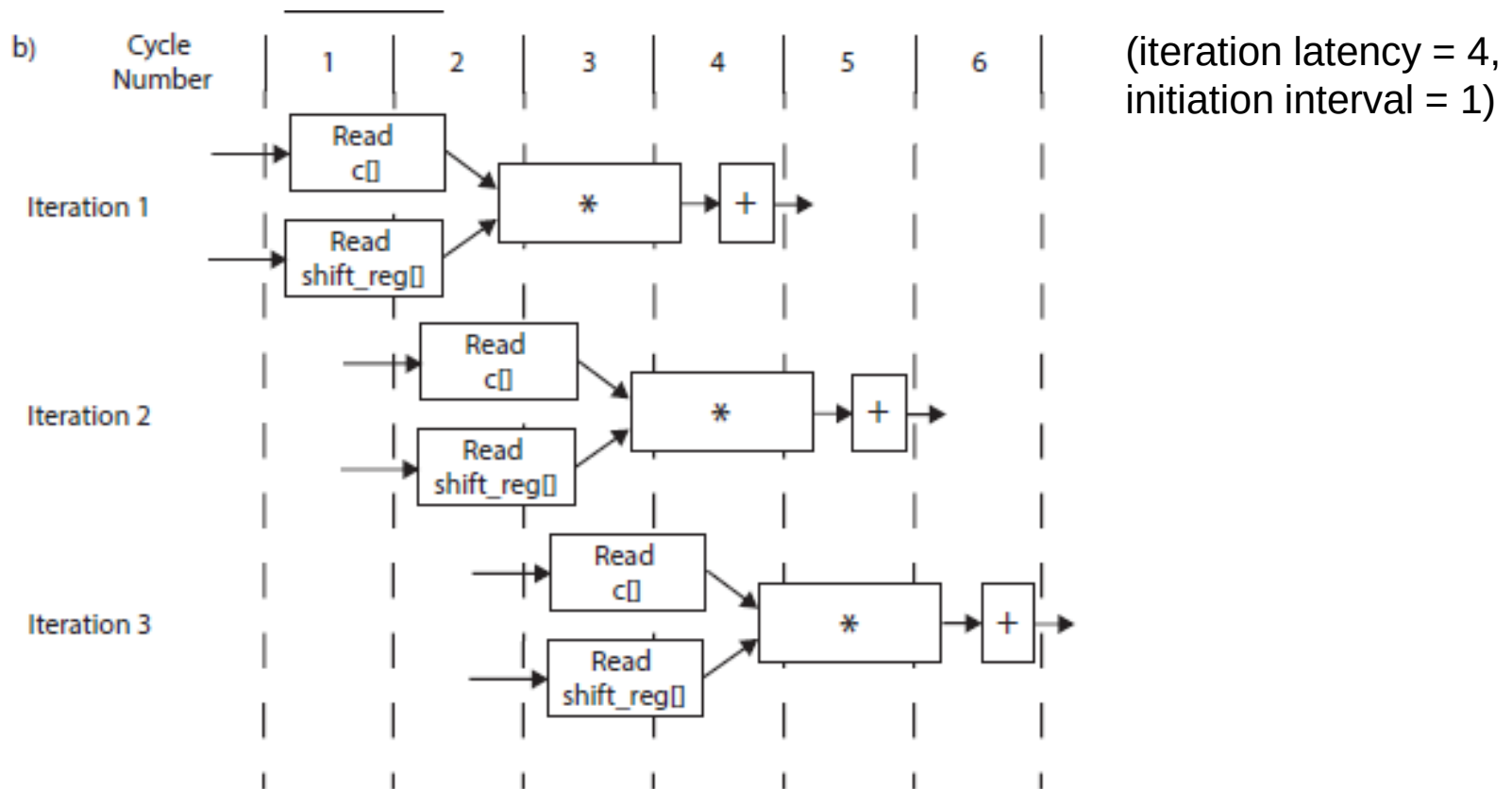
```
for (i = N - 1; i >= 0; i--) {
    if (i == 0) {
        acc += x * c[0];
        shift_reg[0] = x;
    } else {
        shift_reg[i] = shift_reg[i - 1];
        acc += shift_reg[i] * c[i];
    }
}
```

- Non-pipelined (serialized) MAC operations



- Can we start later iterations *before* previous iteration end?

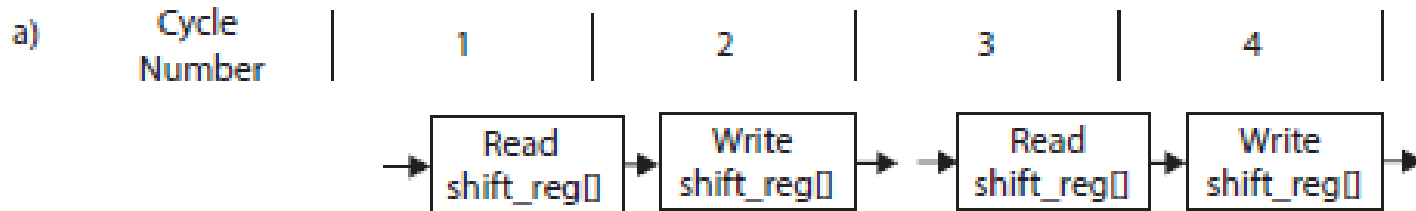
- Loop pipelining: Overlaps the execution of multiple loop iterations
- Pipelined schedule of MAC operation :



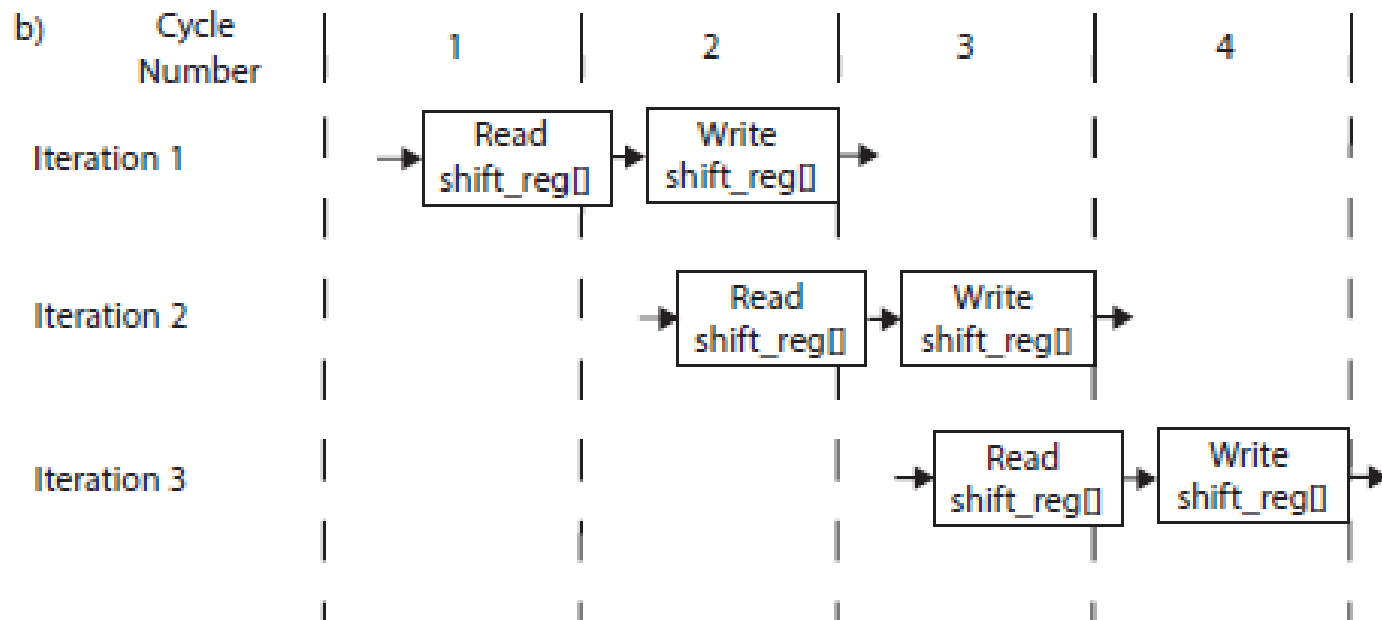
Quiz) How many cycles to process 11 taps?

- Total cycles = $1 * 11 + 3 = 14$ cycles

- Non-pipelined version of shift_reg:



- Pipelined version of shift_reg:



Vitis HLS Pragma : PIPELINE

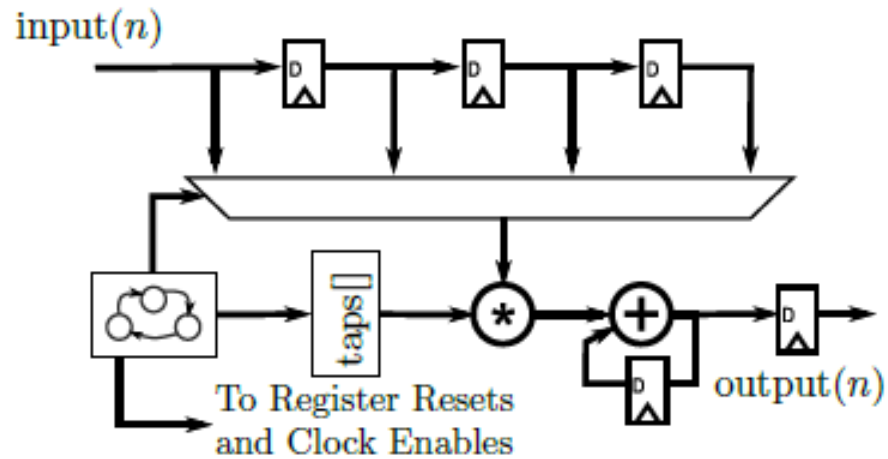
- Syntax: **#pragma HLS PIPELINE II=N**
 - N : Target initiation interval
 - If omitted (#pragma HLS PIPELINE), HLS tool will try to achieve best II possible

– Example:

```
for (i = N - 1; i >= 0; i--){
    #pragma HLS pipeline II=1
    if (i == 0) {
        acc += x * c[0];
        shift_reg[0] = x;
    }
}
```

- Automatic pipelining
 - As default, Vitis HLS will synthesize the loop as if this pragma has been added
 - (a new feature in recent Vitis HLS)

Synthesized Circuit After Pipelining



<one tap per clock version>

Vitis HLS Synthesis Report

- After adding pipeline pragma:

▼ Timing Estimate

Target	Estimated	Uncertainty
4.00 ns	2.893 ns	1.08 ns

▼ Performance & Resource Estimates ⓘ

✓

 Modules

✓

 Loops

Modules & Loops	Issue Type	Violati	Dist	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined
▼ fir	II Violation			-	27	108.000	-	28	-	no
Shift_Accum_	II Violation	Memor 1		-	24	96.000	5	2	11	yes

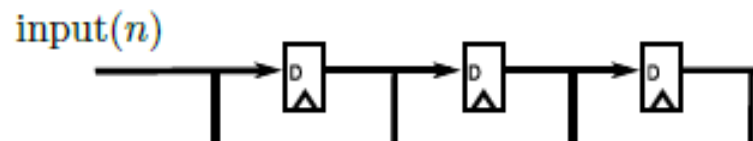
What Happened?

Console Errors Warnings Guidance x Man Pages

16 Guidance-Infos 2 Guidance-Warnings 0 Guidance-Errors

Name	Web Help	Details
▼ All Categories		
▼ SCHEDULE		
⚠ [HLS 200-880]	LINK	The II Violation in module 'fir' (loop 'Shift_Accum_Loop'): Unable to enforce a carried dependner operation ('shift_reg_addr_1_write_ln32', fir.cpp:32) of variable 'shift_reg_load', fir.cpp:32 on ar of variable 'x', fir.cpp:11 on array 'shift_reg'.
i [HLS 200-1470]		Pipelining result : Target II = 1, Final II = 2, Depth = 5, loop 'Shift_Accum_Loop'

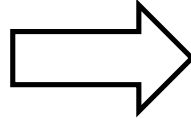
```
Shift_Accum_Loop:
for (i = N - 1; i >= 0; i--){
    if (i == 0) {
        acc += x * c[0];
        shift_reg[0] = x;
    }
    else{
        shift_reg[i] = shift_reg[i - 1];
        acc += shift_reg[i] * c[i];
    }
}
*y = acc;
```



Code Refactoring

- Solution 1: 2.6 Code hoisting

```
for (i = N - 1; i >= 0; i--) {  
    if (i == 0) {  
        acc += x * c[0];  
        shift_reg[0] = x;  
    } else {  
        shift_reg[i] = shift_reg[i - 1];  
        acc += shift_reg[i] * c[i];  
    }  
}
```



```
for (i = N - 1; i > 0; i--) {  
    shift_reg[i] = shift_reg[i - 1];  
    acc += shift_reg[i] * c[i];  
}  
  
acc += x * c[0];  
shift_reg[0] = x;
```

- Solution 2: Use ternary operator

```
for (i = N - 1; i >= 0; i--) {  
    #pragma HLS pipeline II=1  
    shift_reg[i] = (i==0) ? x : shift_reg[i - 1];  
    data_t x_temp = (i==0) ? x : shift_reg[i];  
    acc += x_temp * c[i];  
}
```

Pipelining Result After Optimization

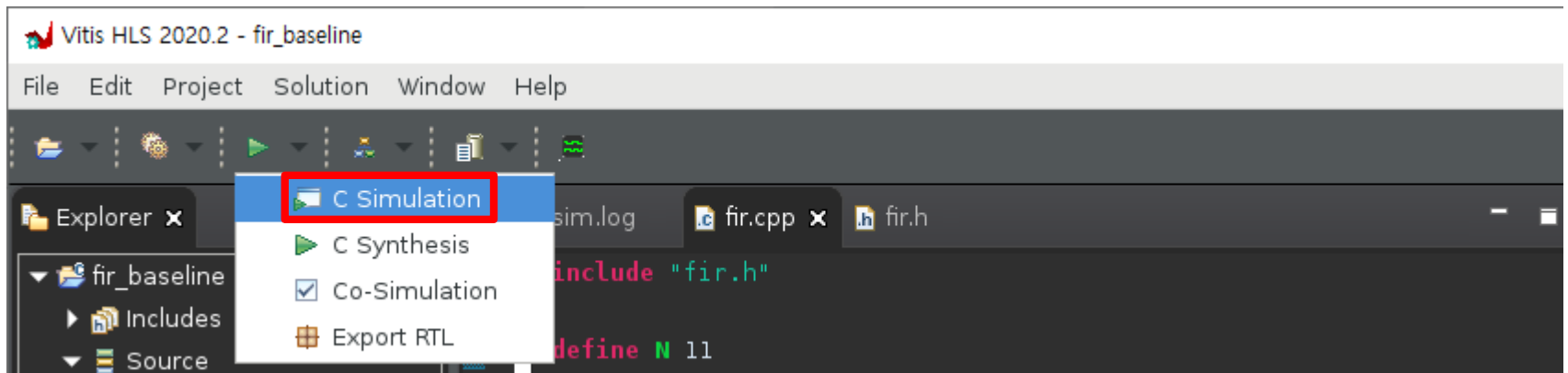
- Synthesis result:

Modules & Loops	Issue Type	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined
▼ fir		-	16	64.000	-	17	-	no
Shift_Accum_Loop		-	14	56.000	5	1	11	yes

- Execution cycles:
 - 600 samples * 17 cycles = 10,200 cycles
- Speedup (from baseline)
 - $16,800 / 10,200 = 1.65X$

Run C simulation

- After code refactoring, always run C simulation to verify correctness:

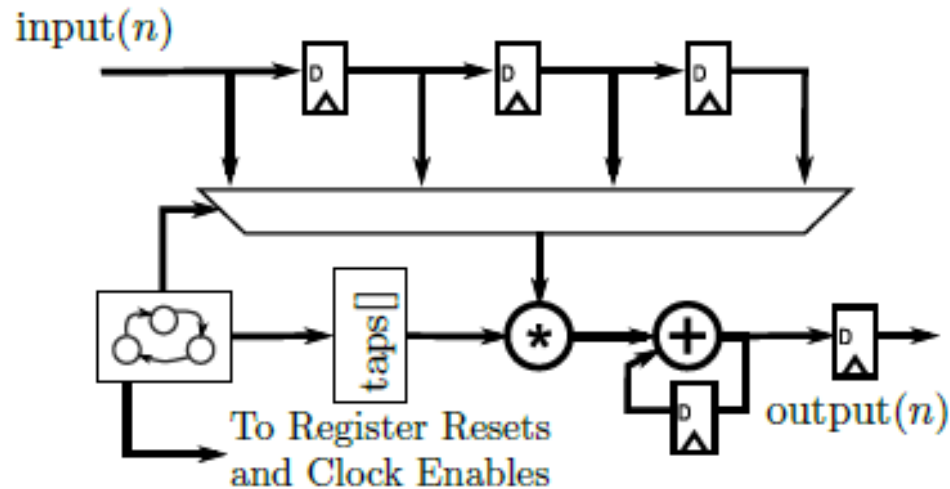


- Result:

```
Comparing against output data
*****
PASS: The output matches the golden output!
*****
INFO: [SIM 211-1] CSim done with 0 errors.
```

Problem with Pipelined FIR

- Q) Any performance problem with pipelined FIR?

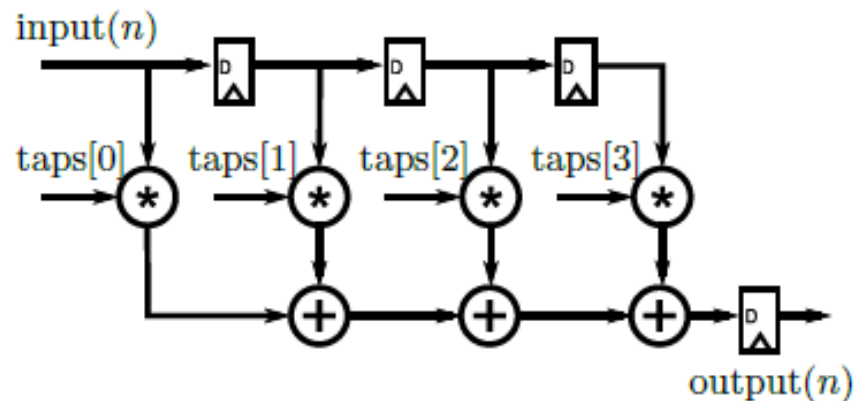


→ Only performs one multiply-and-accumulate per cycle

Can we perform all computation and register shifts in parallel?

2.8 Loop Unrolling

- Loop unrolling: Replication of the loop body
 - Increases parallelism
 - Loop unrolling factor: number of iterations unrolled
- Circuit diagram of a fully unrolled FIR filter:

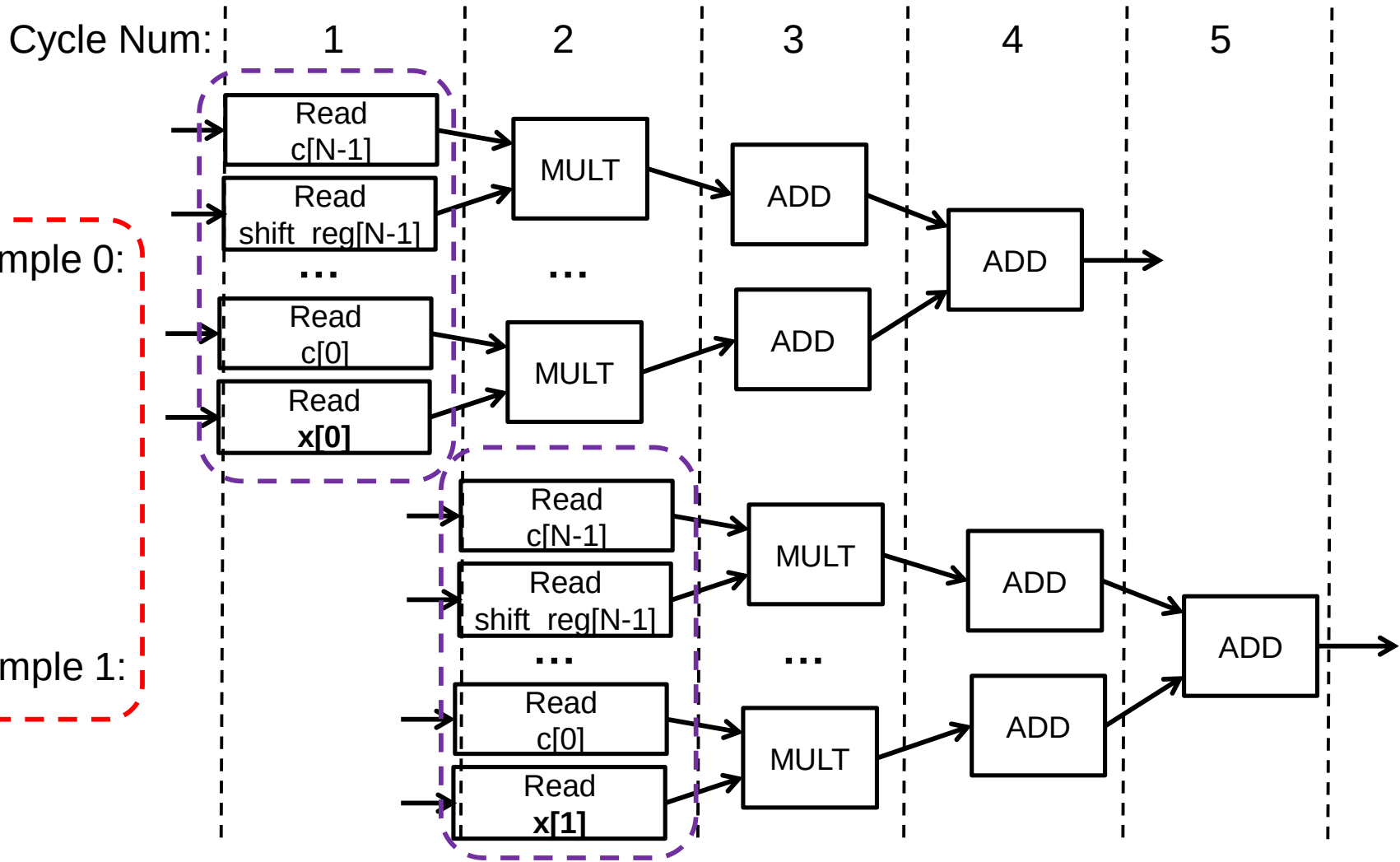


- Processes one sample per clock cycle
- Performs all register shifts, multiplication, addition in parallel

Loop Unrolling vs Loop Pipelining

- Q) What is the difference between loop unrolling and loop pipelining?
 - Loop pipelining: Different types of operators work in parallel (on the different stage of computation)
 - Loop unrolling: Same type of operators work in parallel on different data
- Loop pipelining and loop unrolling are often orthogonal (may be applied at the same time)

- Timing diagram after adding loop unrolling:



The loop on **filter taps** is **unrolled**

The loop on **samples** (and filter taps) is **pipelined**

Vitis HLS Pragma : UNROLL

- Syntax: **#pragma HLS UNROLL FACTOR=N**
 - N : Unrolling factor
 - If omitted, HLS tool will fully unroll the loop

```
for (int i = N - 1; i >= 0; i--){  
#pragma HLS unroll  
    shift_reg[1] = (i==0) ? x[s] : shift_reg[i - 1];  
    data_t x_temp = (i==0) ? x[s] : shift_reg[i];  
    acc += x_temp * c[i];  
}
```

- shift_reg array elements should be made individual registers to allow parallel access
 - Apply array partitioning pragma
- Could have been automated by HLS compiler

```
static data_t shift_reg[N];  
#pragma HLS ARRAY_PARTITION variable=shift_reg complete
```

Exercise

- To do: **Pipeline** the **sample** loop
- Current location of the sample loop (in fir_test.cpp):

```
for (i=0;i<SAMPLES;i++) {  
    fscanf(fin,"%d",&signal);  
    //Call the HLS block  
    fir(&output,signal);  
    // Save the results.  
    fprintf(fp,"%d\n",output);  
    printf("%i %d %d\n",i,signal,output);  
}
```

- Hint: Modify both fir_test.cpp & fir.cpp files, change the location of the sample loop, and put pipeline pragma on the sample loop

Result After Unrolling + Pipelining

- Synthesis result:

Modules & Loops	Issue Type	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	
▼ fir		-	606	2.424E3	-	607	-	no	
VITIS_LOOP_24_1		-	604	2.416E3	6	1	600	yes	

- Execution cycles (after running co-sim):

Performance Estimates ⓘ						
						
Modules	Avg II	Max II	Min II	Avg Latency	Max Latency	Min Latency
● fir				606	606	606

- Speedup (from baseline)
 - $16,800 / 606 = 27.7X$

Summary

- Loop pipelining overlaps the execution of multiple loop iterations
- Loop unrolling replicates the loop body and processes several iterations in parallel
- Loop pipelining and loop unrolling may often be applied at the same time
- Run C simulation after code modification to check correctness